Ligand PathFinder from the command line

A practical guide to `lpf_run.py`, the command-line driver for Ligand PathFinder (LPF) — Rosetta-based ligand and peptide (un)binding pathways.

> Method: Zlobin A, Eisenhuth P, Ferrario V, Meiler J. *Exploration of ligand (un)binding trajectories with Rosetta.* bioRxiv 2026, doi:10.64898/2026.10.07.757317. The upstream code is run **unmodified**; everything here produces input files for it.

Every command below is runnable as written once you substitute your own structures. Numbers quoted as "measured" come from real runs on this set-up, not estimates.

---

1. Before you start

What you need

1. **Rosetta.** Either a compiled `rosetta_scripts` binary, or PyRosetta — `--rosetta-backend auto` (the default) prefers the binary and falls back to an importable PyRosetta, so activating a PyRosetta environment is enough.
2. **A bound pose.** LPF does not find binding poses and neither does this script. Dock or predict the complex first (Rosetta, Vina, GLIDE, AF3, Boltz, chai-1) and look at it.
3. `numpy`, `scipy`, plus `rdkit` for building ligand files from an SD file and `matplotlib` for plots.

Check the set-up

```
python lpf_run.py doctor
```

Reports Python packages, `gzip` / `zip`, the LigandPathFinder checkout, `molfile_to_params.py`, PyRosetta, and any `rosetta_scripts` binaries it can find — then says whether you have a usable backend.

Prove it end to end

```
python lpf_run.py demo --project runs/demo
```

Runs the repository's own butyrylcholinesterase + paraoxon example, short. If the profile starts near −8 REU and climbs, your install is sound — that is the published value for that complex at the catalytic site. Use `--demo-points N` to make it longer or shorter.

---

2. The pipeline

stage	what it does	key output
`prep`	builds `start.pdb`, `lig.params`, `lig_conformers.pdb`, the guide points, the RosettaScripts protocols and the options file	`project.json`
`coarse`	stage 1 — recursive bisection between two points, Rosetta placing the ligand at every node	`coarsepath.txt`
`traj`	stage 2 — spline the guide path, one Rosetta run per window	`interface_scores.txt`
`analyze`	summary, energy profile, CSV, archive	`energy_profile.png`
`cluster`	pool replicates, cluster the paths, average per cluster, convergence	`clusters.png`
`rescore`	recompute interface energies from saved poses, no re-docking	`interface_scores_rescored.txt`

`prep` writes `project.json`; every later stage reads it. That is what lets each stage be its own job on a cluster.

`run` does `prep` + `coarse` + `traj` + `analyze` in one go.

---

3. Quick starts by system type

3.1 Small molecule, bound pose in an SD file

The common case. The SD file's **first entry must be the bound pose** — `molfile_to_params` writes entry 1 to `lig.pdb`, and that is what gets merged with the protein.

```
python lpf_run.py run \
    --target complex_or_apo.pdb --target-chains A \
    --ligand ligand.sdf \
    --project runs/my_ligand
```

Hydrogens are added automatically if your SD file has none (PyMOL exports usually do not) — Rosetta needs them, and without them every energy is wrong.

3.2 Small molecule, params you already made

If you followed the upstream tutorial and have `lig.params` + `lig_conformers.pdb`:

```
python lpf_run.py run \
    --target complex.pdb --target-chains A \
    --ligand-dir my_ligand_files/ \
    --ligand-pose complex.pdb --ligand-pose-chain X \
    --project runs/mine
```

`--ligand-pose` is a PDB containing the ligand in its bound pose with atom names matching `lig.params`; `--ligand-pose-chain` says which chain of it to take when the file is a whole complex.

3.3 Small molecule from SMILES (testing only)

```
python lpf_run.py run --target protein.pdb --target-chains A \
    --ligand-smiles "CCOP(=O)(OCC)Oc1ccc(cc1)[N+](=O)[O-]" \
    --project runs/smoke
```

This has **no bound pose** — the geometry is a free-solution guess. Fine for checking the machinery, useless as science.

3.4 Protein–peptide

The peptide comes out of the complex it is bound in and becomes one Rosetta ligand residue:

```
python lpf_run.py run --mode peptide \
    --target complex.pdb --target-chains A --peptide-chain B \
    --project runs/my_peptide
```

`--mode peptide` switches the parameter defaults, the protocol tuning and the conformer strategy. See §9.

Variants:

```
--peptide-pdb pep.pdb           # peptide already in its own file
--peptide-sequence PPPALPPKK    # override the sequence read from the PDB
--no-repair-terminus            # do not build a missing terminal O/OXT
--no-protonation                # keep neutral termini instead of pH-7 states
```

3.5 A cofactor or metal in the system

Keep the cofactor's chain, keep its residue name, and give Rosetta its params:

```
python lpf_run.py run \
    --target p450_complex.pdb --target-chains A,B \
    --keep-hetero HEM --extra-params files/HEM.params \
    --ligand-dir ligand/ --ligand-pose ligand.pdb \
    --project runs/p450
```

`--extra-params` puts the file in `extra_files` **and** in the options file's `-extra_res_fa`, which are both required. For metals add `--auto-setup-metals`, and for metal-binding constraint records use `--header mymetals.txt` (prepended verbatim to every `task.pdb`).

3.6 An external CAVER / AQUA-DUCT path

Skip stage 1 entirely:

```
python lpf_run.py run \
    --target protein.pdb --target-chains A --ligand lig.sdf \
    --guide-path caver_tunnel.pdb \
    --preset benchmark \
    --project runs/tunnel
```

Text `x y z` per line or a PDB both work. Requirements:

- **at least 4 points**, because stage 2 fits a cubic smoothing spline;
- it must run **from the binding site outwards**. Both stages overwrite the path's first point with the ligand's real position, so a tunnel exported outside-in is silently mangled. That is checked and refused; add `--reverse-path` to flip it.
- if your path starts far from the ligand on purpose, add `--dock-zero` so the ligand is docked onto the path's first point before the run starts.

`--preset benchmark` uses the parameters from the paper's own 136-system benchmark (smoothing 5.0, which suits an externally computed tunnel better than the demo's 0.5).

---

4. Stage by stage

4.1 `prep` — everything before Rosetta runs

```
python lpf_run.py prep --project runs/mine \
    --target complex.pdb --target-chains A \
    --ligand ligand.sdf \
    --dry-run
```

`--dry-run` builds and checks every input but never calls Rosetta. Use it first on a new system; it costs seconds and catches most mistakes.

What to read in its output:

- **the protein inventory** — chains kept, waters removed, hetero residues retained;
- `lig.params` — atom count, and whether hydrogens were added;
- **the tracked atom** — `NBR_ATOM`, which is what follows the guide path;
- **conformer subsampling preview** — how many rotamers Rosetta actually gets. If it says 1–2, your ligand is effectively rigid;
- **the exit scan** — ranked routes with their bottleneck widths;
- `SCORINGGRIDS width` — must contain the molecule.

Useful options:

```
--target 3HTB                # a 4-character PDB ID is downloaded
--keep-waters                # keep crystallographic waters
--strip-hydrogens            # drop H and let Rosetta rebuild them
--ligand-chain P             # if chain X is taken; the XMLs are retargeted
--resname LIG                # 3-character PDB residue name
--n-conformers 400           # bigger ETKDG library
--prune-rmsd 0.3             # keep conformers closer together
--nbr-central                # track the most central atom, not molfile's choice
--nbr-atom C17               # or name it directly
--minimize-start             # relax the complex into the force field first
```

4.2 Choosing the exit route

By default `prep` casts rays from the bound pose and ranks them by how wide the exit is. To see the options without committing:

```
python lpf_run.py prep --project runs/mine \
    --target complex.pdb --target-chains A --ligand ligand.sdf \
    --list-rays 12
```

prints the top 12 and stops. Then pick one:

```
--ray-index 3                # take the 4th-ranked ray
--hint-vector 0,1,0 --hint-cone 45   # only scan within 45 deg of +y
--exit-point 9.2,-25.5,-1.3  # skip the scan, name the point
--min-exit-distance 30       # force a longer path
--exit-clearance 10          # endpoint must be 10 A clear of the protein
```

The **bottleneck** column is the narrowest clearance along the ray. Compare it with your ligand's radius: if the bottleneck is much smaller, the protein has to open for the ligand to pass, which is legitimate but makes the run harder.

4.3 Relaxing the start pose

The upstream tutorial (§1.3) requires the complex to be relaxed into the Rosetta force field before LPF runs. This is **not** what window 0 does — that only settles the interface region.

```
python lpf_run.py prep --project runs/mine \
    --target complex.pdb --target-chains A --ligand ligand.sdf \
    --minimize-start --minimize-nstruct 5 --minimize-ca-sd 0.5
```

Runs N minimisations and keeps the best by constraint-free `total_score`. `--minimize-ca-sd 0` reproduces the tutorial's unrestrained `minimize` exactly; the default adds C-alpha coordinate restraints so the fold cannot drift. Either way, **look at the result** — this is the step most likely to need system-specific attention.

4.4 `coarse` — stage 1

```
python lpf_run.py coarse --project runs/mine --replicates 10
```

Recursive bisection between the two guide points. It is stochastic and jitters its endpoint, so several replicates then the medoid is the recommended practice — `--replicates N` does that and records the medoid for stage 2.

```
--coarse-step 2.0             # node spacing (3.0 for peptides)
--coarse-nstruct 10           # decoys per node
--jitter-radius 5.0           # endpoint jitter disc
--keep-previous               # do not wipe an existing coarse directory
```

Skip this stage entirely if you supplied `--guide-path` with 4+ points.

4.5 `traj` — stage 2, the expensive one

```
python lpf_run.py traj --project runs/mine
```

Splines the guide path, resamples at `--traj-step`, and runs Rosetta once per window, each starting from the previous window's result. Resumes from `checkpoint.pkl` if interrupted — just run the same command again.

```
--traj-step 0.5               # window spacing (1.0 for peptides)
--traj-smooth 0.5             # spline smoothing (5.0 for CAVER paths)
--traj-nstruct 10             # decoys per window
--run-stats --nstruct-stats 10 # second scoring pass = error bars per window
--outlier-rmsd 2.0            # reject decoys that drifted this far
--end-padding 5               # relax the in-plane restraint near the end
--noise                       # jitter the projected point, escapes local traps
--memory-mode / --no-memory-mode   # carry protein relaxation forward or not
--dock-zero                   # dock onto the path's first point before starting
--checkpoint-interval 10      # 0 disables
--use-tmp --tmp-path /scratch # run each window on fast storage
--fresh                       # ignore the checkpoint and start over
--run-name repl-07            # name the run subdirectory
```

What to watch in the output:

- **the curvature warning at start-up** — the maximum turning angle of the guide path, and what 2× / 5× smoothing would give. Above ~25° the angle constraint may not hold direction;
- `cst:` per window — constraint energies. Large and growing means the ligand is fighting the restraints;
- `d_to_plane` / `d_in_plane` — how far the tracked atom ended from the guide plane and point. Small means the constraints are holding;
- `SKIPPED` — windows where the guide path folds back (`skip` is on).

4.6 `analyze`

```
python lpf_run.py analyze --project runs/mine --plot --csv --zip
```

```
--run-dir runs/mine/repl-03   # a specific run
--plot profile.png            # name the image
--csv profile.csv             # window, distance, interface, total
--smooth-window 5             # also draw a running mean
--zip --zip-windows           # archive, optionally including every window
```

---

5. Replicates, convergence and clustering

A single replicate is noisy; the paper is explicit about that.

Running them

```
# serial, one flag
python lpf_run.py traj --project runs/mine --traj-replicates 30

# parallel on a workstation: 6 at a time
printf 'repl-%02d\n' $(seq 0 29) | xargs -P 6 -I{} \
    python lpf_run.py traj --project runs/mine --run-name {}
```

Budget 1–2 GB of RAM per concurrent process with the PyRosetta backend. `LPF_SHIM_JOBS` parallelises decoys *within* a window instead — do not use both at once.

On a cluster, an sbatch array is the same idea, scheduled:

```
#!/bin/bash
#SBATCH -n 1 --cpus-per-task=1 --mem=5G -t 720
python lpf_run.py traj --project /path/to/project \
    --run-name repl-$SLURM_ARRAY_TASK_ID
```

```
sbatch -a 0-29 run.bash
```

Same guide path, or one each?

All replicates share the medoid path by default, which is what the paper's own sbatch array does (it copies one `run.json` into every replicate directory). The coarse stage is where routes are explored. To sample routes at the trajectory level instead:

```
python lpf_run.py coarse --project runs/mine --replicates 10
python lpf_run.py traj   --project runs/mine --traj-replicates 10 \
                         --path-per-replicate
```

	measures
shared path (default)	the uncertainty of the profile **along one route**
`--path-per-replicate`	which routes exist and how their energetics compare

Replicates still differ on a shared path: the in-plane well is `ipdc_d0` wide, the docking is stochastic, `memory_mode` accumulates induced fit differently, and `skip` depends on where the ligand actually went.

Pooling and judging

```
python lpf_run.py cluster --project runs/mine --convergence --csv
```

Clusters the paths by frame-weighted ligand RMSD, hands the negative distance matrix to affinity propagation, averages the interface profile per cluster, and discards clusters of 10 or fewer — the paper's recipe. `--convergence` additionally bootstraps subsets and prints how much the pooled mean still moves:

```
  n    spread of the mean (REU)
  2                     0.757
  5                     0.482
 12                     0.310
 30                     0.201
```

To resolve a feature of size *D* REU you want the spread comfortably below *D*. It falls as 1/sqrt(n), drawn as a reference on the plot. **A curve sitting above the reference means the replicates disagree structurally** — different exit routes — and the answer is to cluster them, not to average harder.

```
--min-cluster-size 10         # the paper's value; lower it for small ensembles
--missing-penalty 0.5         # distance charged per missing frame
--frame-step 0.33             # x-axis spacing; default from run.json
runs/a runs/b runs/c          # or name the directories / a glob explicitly
```

How many do you need? The paper's tutorial runs **30** in its worked example and its clustering discards clusters of 10 or fewer, so an ensemble meant for clustering must be well above 10. The SI does not state the per-system counts behind the published figures.

---

6. When something goes wrong

`interface_scores.txt` is all zeros

Not a result. Rosetta's `InterfaceScoreCalculator` contributed nothing and the column was written as a placeholder. The poses are fine, so recompute without re-docking:

```
python lpf_run.py rescore --project runs/mine --install
```

One scoring call per window instead of `nstruct` docking runs — **measured at 188 s for 43 windows**. `--install` replaces `interface_scores.txt` (backing up the original) so plots and clustering pick the

values up.

With the PyRosetta backend this is expected on some builds: confirmed on PyRosetta 2024.19, the mover delivers nothing readable from the pose even when constructed directly in Python. The shim therefore computes the interface energy itself — complex minus separated, constraint terms zeroed — which agrees with Rosetta's own `InterfaceAnalyzerMover` to three decimals. To inspect it:

```
cd runs/mine/trajectory && unzip -q window_04.zip && cd window_04
mkdir -p grid
python ../../../../pyrosetta_rosetta_scripts.py \
    --probe task.xml task.pdb --options options
```

Lists the pose's chains, every score term it carries, and runs the computation directly so you can see the number.

Other common stops

symptom	cause and fix
`no rosetta_scripts binary found`	the default `auto` backend falls back to PyRosetta; on an older copy pass `--rosetta-backend pyrosetta`
`the trajectory stage needs at least 4 guide points`	run `coarse` first, or supply a denser `--guide-path`
`this path appears REVERSED`	the tunnel was exported outside-in; add `--reverse-path`
`the path starts N A from the ligand and dock_zero is off`	add `--dock-zero`, or re-anchor the path at the ligand
`ligand is effectively rigid in this window`	raise `--conformer-threshold`, or build a bigger library
`bottleneck X A is tight for a molecule spanning Y A`	the protein must open; keep `memory_mode` on, or try another `--ray-index`
run stalls or the ligand cuts a corner	raise `--traj-smooth`, or take a straighter coarse medoid
`n_proc > 1 needs an .mpi. build`	use the MPI binary, or `--shim-jobs` with PyRosetta
ligand too stiff / too floppy	the retuning checklist in the upstream `demo/tutorial.md` §5

Moving a project directory

Every path this tooling writes is absolute — deliberately, so a run behaves the same however it is launched — which means moving a project afterwards leaves stale paths in `project.json`, each run's `run.json`, the checkpoints and the PyRosetta launcher. Move it, then repair it:

```
mv runs/mine work/runs/mine
python lpf_run.py relocate --project work/runs/mine --dry-run
python lpf_run.py relocate --project work/runs/mine
```

It reads the old location out of `project.json` itself, so you do not have to remember it. For files that lived *outside* the project — the structure you passed to `--target`, say — add a mapping:

```
python lpf_run.py relocate --project work/runs/mine \
    --remap /old/path/3HTB.pdb=/new/path/3HTB.pdb
```

`.log` files keep the old paths on purpose: they record where the run actually happened.

Overriding anything

Two escape hatches:

```
--param rmsd_constraint_sd=1.5    # any parameters.* key, repeatable
--set output.use_tmp=true         # passed straight to LPF as --set
```

`--param` edits the parameter set this script builds; `--set` is handed to the upstream script untouched. Use `--param` unless you need a key outside `parameters`.

---

7. Worked example: T4 lysozyme L99A/M102Q + JZ4 (PDB 3HTB)

A complete session on a real system, with the numbers it actually produced.

```
cd /path/to/lpf_notebooks

# 1. check the backend
python lpf_run.py doctor

# 2. prepare: receptor chain A, ligand from an SD file.
#    JZ4 exported from PyMOL has no hydrogens; they are added automatically
#    (10 -> 22 atoms), which also takes the conformer library from 2 to 112.
python lpf_run.py prep --project runs/3HTB \
    --target 3HTB --target-chains A \
    --ligand ligand.sdf \
    --dry-run

# 3. the same without --dry-run once the report looks right
python lpf_run.py prep --project runs/3HTB \
    --target 3HTB --target-chains A --ligand ligand.sdf

# 4. stage 1, a few replicates, medoid forward
python lpf_run.py coarse --project runs/3HTB --replicates 5

# 5. stage 2 — measured at ~15 s per window, 43 windows, so ~11 min
python lpf_run.py traj --project runs/3HTB --run-name repl-00

# 6. look at it
python lpf_run.py analyze --project runs/3HTB \
    --run-dir runs/3HTB/repl-00 --plot --csv

# 7. scale up: 29 more, six at a time, about an hour
printf 'repl-%02d\n' $(seq 1 29) | xargs -P 6 -I{} \
    python lpf_run.py traj --project runs/3HTB --run-name {}

# 8. pool, cluster, and check whether 30 was enough
python lpf_run.py cluster --project runs/3HTB --convergence --csv
```

What one replicate gave:

```
bound (window 0)   −11.41 REU
deepest minimum    −12.24 REU  (window 1)
secondary minimum   −9.4  REU  (windows 15–18)
final               −1.10 REU
released           +11.1  REU
```

The secondary minimum is the ligand pausing against a second site on the way out — the kind of feature this method exists to find. Whether it is real is a question for the replicate ensemble, not one run.

---

8. What things cost

Measured on a single core, `nstruct` 10 unless noted.

system	ligand	protein	per window	per 40-window replicate
T4 lysozyme + JZ4	22 atoms	163 res	**~15 s**	~10 min
BChE + paraoxon (upstream demo)	32 atoms	526 res	~42 s	~28 min
SH3 + 9-mer peptide	146 atoms	57 res	~85 s	~57 min
the paper's 136-system benchmark	41 atoms avg	mixed	~2.7 min avg (0.9–8.2)	~1.8 h

Rough scaling: cost grows roughly linearly with ligand atom count, and with how much of the protein the `LigandArea` cutoffs let move. `--sec-per-decoy` feeds the estimate printed before each stage; recalibrate it from your own first window and the estimate becomes reliable.

A 30-replicate ensemble is therefore anywhere from 5 h (small ligand, small protein) to several days of core time. Parallelise across replicates, not within windows, unless you are memory-rich.

---

9. Peptides

The peptide becomes **one Rosetta ligand residue**. All of its backbone and side-chain freedom has to live in the conformer library, because the options file sets `-packing:ignore_ligand_chi true` and Rosetta will not sample those torsions on the fly.

`--mode peptide` therefore:

- extracts the chain and reports missing heavy atoms per residue;
- rebuilds a truncated terminal carboxyl (planar `O` / `OXT`) but refuses to invent missing side chains — fix those upstream;
- transfers bond orders from a sequence template and sets pH-7 protonation (`-COO⁻`, Lys `-NH₃⁺`, Arg guanidinium, charged N-terminus including proline);
- builds the library as a **connected ladder** from the bound pose — local perturbations, medium perturbations, and free embeddings — and reports whether it is actually connected;
- re-picks `NBR_ATOM` to a central backbone atom;
- sizes the scoring grid from the molecule and applies the paper's >50-atom rule (`conformer_rmsd_threshold` and `outlier_rmsd` both 5.0).

Library controls:

```
--n-local 120 --sigma-local 12      # small perturbations of a few torsions
--n-medium 120 --sigma-medium 45    # larger perturbations of more torsions
--n-global 60                       # free ETKDG embeddings
--prune-rmsd 0.6                    # minimum separation between kept conformers
--conformer-report                  # print the ladder and its connectivity
```

Read the connectivity report. If it says `NO`, the peptide can never walk from its bound conformation to an extended one, so it leaves as a near-rigid body and the profile describes something that cannot

happen. Raise `--n-medium` and `--sigma-medium`, lower `--prune-rmsd`, or raise `--conformer-threshold`.

How long a peptide?

Measured, 9 residues versus 20:

	9-mer (verified working)	20-mer
atoms	146	305
rotatable torsions	23	82
span over rotamers	34 Å	43 Å
scoring grid	56 Å	64 Å — the cap
`NBR_RADIUS` as generated	23 Å	48 Å (25 Å after re-picking the central atom)
library connected?	yes at 5 Å	**no, at any threshold up to 8 Å**
per window, `nstruct` 10	~85 s	~3–6 min

2–12 residues is the usable range. At 20 the cost is survivable but the conformational coverage is not: ~80 conformers cannot represent a 20-residue chain, and more conformers do not close the gap because the space grows far faster. Beyond a dozen residues the question has become protein–protein unbinding, which needs real backbone degrees of freedom rather than a rotamer library.

---

10. Files a run produces

Inside the project directory:

```
project.json                what prep decided; every later stage reads it
protein.pdb                 the cleaned receptor
start.pdb                   receptor + ligand, the layout LPF requires
start_minimized.pdb         if --minimize-start was used
peptide.pdb                 the repaired peptide, in peptide mode
conformers.sdf              the library fed to molfile_to_params
ligand/lig.params           the Rosetta residue type
ligand/lig_conformers.pdb   the rotamer library
files/*.xml, files/options  protocols retargeted to your chain and size
path                        the two guide points (or your supplied path)
header.pdb                  prepended to every task.pdb; usually empty
coarse_00/ .. coarse_NN/    stage 1 runs; coarsepath.txt in each
repl-00/ .. repl-NN/        stage 2 replicates
pyrosetta_backend/          the launcher, with the PyRosetta backend
```

Inside a trajectory run directory:

file	what it is
`interface_scores.txt`	`interface_delta_X` per scored window — **the result**
`nbr_trajectory.txt`	the tracked atom's position per window
`full_trajectory.pdb`	protein + ligand, one frame per window
`lig_trajectory.pdb`	the ligand alone
`total_scores.txt`	whole-complex energy, mostly for debugging
`stats.pkl`	per-window score distributions, if `--run-stats`
`guide_path`, `spline`, `normals`, `input_path`	path construction intermediates
`checkpoint.pkl`	resume state
`window_XXXX/` or `.zip`	per-window working directories

Open a trajectory in PyMOL with:

```
pymol runs/mine/repl-00/full_trajectory.pdb
# then:  set all_states, off ;  mset 1 x43 ;  mplay
```

---

11. Complete option reference

Generated from the parser itself, so it cannot drift from the code. Options shared by several stages are listed under each.

`lpf_run.py prep`

Build start.pdb, the ligand files, the guide points, the protocols and the options file, and write project.json.

project

option	value	default	what it does
`--project`, `-o`	`DIR`		project directory: holds the prepared inputs and every run
`--n-proc`	`N`	`1`	MPI ranks per Rosetta call; >1 needs an .mpi. build and N <= nstruct (default: 1)
`--no-archive`	flag		do not zip each finished node/window directory
`--dry-run`	flag		prepare and preflight, but never call Rosetta
`--sec-per-decoy`	`S`	`16.0`	only for the runtime estimate. The default is implied by the paper's benchmark (2.7 min/frame at nstruct 10, one core, 136 systems); its spread is 0.9-8.2 min/frame, so recalibrate from your own first window (default: 16)
`--set`	`KEY=VALUE`		passed straight to LPF as --set (repeatable), e.g. --set output.use_tmp=true *(repeatable)*
`--verbose`, `-v`	flag		more detail from the preparation steps

environment

option	value	default	what it does
`--lpf-repo`	`DIR`		LigandPathFinder checkout (searched for if omitted)
`--clone`	flag		clone LigandPathFinder if it is not found

Rosetta

option	value	default	what it does
`--rosetta`	`PATH`		rosetta_scripts binary (searched for if omitted)
`--rosetta-database`	`DIR`		Rosetta database; found next to the binary if omitted. Written into the options file as -in:path:database
`--rosetta-backend`	`auto` \	`binary` \	`pyrosetta`
`--pyrosetta-python`	`PATH`		interpreter that has PyRosetta (default: this one)
`--install-pyrosetta`	flag		pip-install PyRosetta if absent (~1.5 GB download)
`--shim-jobs`	`N`	`1`	PyRosetta backend: worker processes per Rosetta call; each needs 1-2 GB RAM (default: 1)

target protein

option	value	default	what it does
`--target`, `-t`	`PDB	ID`		
`--target-chains`	`A,B`		protein chains to keep (default: all)
`--keep-hetero`	`HEM,ZN`		hetero residues to keep; each needs its own .params via --extra-params
`--keep-waters`	flag		keep water molecules
`--strip-hydrogens`	flag		drop hydrogens and let Rosetta rebuild them
`--extra-params`	`F.params`		cofactor params files, added to extra_files and to -extra_res_fa
`--auto-setup-metals`	flag		add -in:auto_setup_metals to the options file
`--header`	`FILE`		lines prepended to every task.pdb (metal constraints, membrane spans, symmetry)

ligand / peptide

option	value	default	what it does
`--mode`	`ligand \`	`peptide`	`ligand`
`--ligand` , `-l`	`FILE.sdf`		SD/MOL file whose FIRST entry is the bound pose
`--ligand-smiles`	`SMILES`		build from SMILES — no bound pose, so for testing only
`--ligand-dir`	`DIR`		directory that already holds lig.params + lig_conformers.pdb
`--ligand-pose`	`PDB`		with --ligand-dir: the ligand in its bound pose, atom names matching lig.params
`--ligand-pose-chain`	`CH`		if --ligand-pose is a whole complex, which chain in it is the ligand
`--peptide-chain`	`CH`		--mode peptide: chain of the bound peptide in --target
`--peptide-pdb`	`PDB`		--mode peptide: peptide-only PDB, in the receptor's frame
`--peptide-sequence`	`SEQ`		override the one-letter sequence read from the PDB
`--no-repair-terminus`	flag		do not build a missing terminal O/OXT
`--no-protonation`	flag		keep the neutral template instead of pH-7 charge states
`--ligand-chain`	`CH`	`X`	chain the ligand gets in start.pdb; must be free (default: X)
`--ligand-resnum`	`N`	`1`	residue number of the ligand in its chain (default: 1)
`--resname`	`ABC`		3-character PDB residue name (default: lig, or pep for --mode peptide)

conformer library

option	value	default	what it does
`--n-conformers`	`N`	`200`	ligand mode: ETKDG conformers to embed (default: 200)
`--n-local`	`N`	`120`	peptide mode: small perturbations of a few torsions
`--n-medium`	`N`	`120`	peptide mode: larger perturbations of more torsions
`--n-global`	`N`	`60`	peptide mode: free ETKDG embeddings
`--sigma-local`	`DEG`	`12.0`	
`--sigma-medium`	`DEG`	`45.0`	
`--prune-rmsd`	`A`		minimum RMSD between kept conformers (default: 0.5 ligand, 0.6 peptide)
`--no-minimize`	flag		skip the short MMFF relaxation of each conformer
`--kekulize`	flag		write a Kekule SD file; molfile_to_params expects explicit aromatic bonds, so leave this off
`--recharge`	`CHG`		force the net charge instead of taking it from the SD file's formal charges
`--conformer-report`	flag		report the library's RMSD ladder and whether it is connected
`--molfile-to-params`	`PATH`		Rosetta's molfile_to_params.py (searched for, then downloaded, if omitted)
`--no-download`	flag		never download molfile_to_params.py
`--seed`	`N`	`61453`	

tracked atom (NBR_ATOM)

option	value	default	what it does
`--nbr-atom`	`NAME`		params atom name to track, e.g. C17
`--nbr-atom-source`	`"CA 5"`		name it in the original peptide instead, e.g. "CA 5"
`--nbr-central`	flag		ligand mode: pick the most central atom rather than keeping molfile_to_params' choice (always done for peptides)
`--nbr-radius-margin`	`A`	`0.5`	

guide points

option	value	default	what it does
`--guide-path`	`FILE`		your own path (xyz text or PDB), e.g. from CAVER or AQUA-DUCT. With 4+ points you can skip stage 1. It must run from the binding site outwards
`--reverse-path`	flag		flip --guide-path, for a tunnel exported outside-in
`--exit-point`	`X,Y,Z`		exit point, instead of scanning for one
`--ray-index`	`N`	`0`	which ranked exit ray to take (default: 0, the widest)
`--list-rays`	`N`	`0`	print the top N exit rays and stop
`--hint-vector`	`X,Y,Z`		restrict the scan to a cone about this direction
`--hint-cone`	`DEG`	`60.0`	
`--scan-directions`	`N`	`512`	
`--scan-max-dist`	`A`	`50.0`	
`--exit-clearance`	`A`	`8.0`	how far from the protein the endpoint must sit (default: 8)
`--min-exit-distance`	`A`		path length floor (default: from the molecule's span)

start-pose minimisation (tutorial 1.3)

option	value	default	what it does
`--minimize-start`	flag		relax the assembled complex into the Rosetta force field before running LPF. Do this unless your start pose is already minimised — LPF assumes it is
`--minimize-nstruct`	`N`	`5`	independent minimisation attempts; the best by constraint-free total_score is kept (default: 5)
`--minimize-ca-sd`	`A`	`0.5`	C-alpha coordinate restraint sd; 0 reproduces the tutorial's unrestrained `minimize` run (default: 0.5)
`--no-minimize-backbone`	flag		side chains and rigid-body only

protocols

option	value	default	what it does
`--grid-width`	`A`		SCORINGGRIDS width (default: sized from the molecule)
`--transform-box`	`A`		Transform translation box (default: 5 ligand, 8 peptide)
`--no-tuning`	flag		copy the upstream protocols verbatim, only retargeting the chain

`lpf_run.py coarse`

Needs a project prepared by `prep`.

project

option	value	default	what it does
`--project`, `-o`	`DIR`		project directory: holds the prepared inputs and every run
`--n-proc`	`N`	`1`	MPI ranks per Rosetta call; >1 needs an .mpi. build and N <= nstruct (default: 1)
`--no-archive`	flag		do not zip each finished node/window directory
`--dry-run`	flag		prepare and preflight, but never call Rosetta
`--sec-per-decoy`	`S`	`16.0`	only for the runtime estimate. The default is implied by the paper's benchmark (2.7 min/frame at nstruct 10, one core, 136 systems); its spread is 0.9-8.2 min/frame, so recalibrate from your own first window (default: 16)
`--set`	`KEY=VALUE`		passed straight to LPF as --set (repeatable), e.g. --set output.use_tmp=true *(repeatable)*
`--verbose`, `-v`	flag		more detail from the preparation steps

shared stage parameters

option	value	default	what it does
`--conformer-threshold`	`A`		conformer_rmsd_threshold; also switches subsampling on
`--memory-mode`	flag		carry the whole relaxed pose forward, so induced protein motion accumulates
`--no-memory-mode`	flag	`true`	transplant only the ligand coordinates each step
`--dock-zero`	flag		dock onto the path's first point before starting
`--param`	`KEY=VALUE`		override any parameters. *key (repeatable), e.g. --param rmsd_constraint_sd=1.5* (repeatable)*
`--preset`	`auto` \	`benchmark`	`auto`
`--no-large-ligand-rule`	flag		do not apply the paper's >50-atom settings (conformer_rmsd_threshold and outlier_rmsd both 5.0)

stage 1 · coarse

option	value	default	what it does
`--coarse-step`	`A`		node spacing (default: 2.0 ligand, 3.0 peptide)
`--coarse-nstruct`	`N`	`10`	decoys per node (default: 10)
`--replicates`, `-r`	`N`	`1`	independent coarse runs; the medoid path goes forward (default: 1)
`--jitter-radius`	`A`		endpoint jitter radius (default: 5)
`--keep-previous`	flag		do not wipe an existing coarse directory (the coarse stage cannot resume, so re-runs normally need a clean one)

`lpf_run.py traj`

Needs a project prepared by `prep` (and normally a coarse path). Resumes from checkpoint.pkl.

project

option	value	default	what it does
`--project`, `-o`	`DIR`		project directory: holds the prepared inputs and every run
`--n-proc`	`N`	`1`	MPI ranks per Rosetta call; >1 needs an .mpi. build and N <= nstruct (default: 1)
`--no-archive`	flag		do not zip each finished node/window directory
`--dry-run`	flag		prepare and preflight, but never call Rosetta
`--sec-per-decoy`	`S`	`16.0`	only for the runtime estimate. The default is implied by the paper's benchmark (2.7 min/frame at nstruct 10, one core, 136 systems); its spread is 0.9-8.2 min/frame, so recalibrate from your own first window (default: 16)
`--set`	`KEY=VALUE`		passed straight to LPF as --set (repeatable), e.g. --set output.use_tmp=true *(repeatable)*
`--verbose`, `-v`	flag		more detail from the preparation steps

shared stage parameters

option	value	default	what it does
`--conformer-threshold`	`A`		conformer_rmsd_threshold; also switches subsampling on
`--memory-mode`	flag		carry the whole relaxed pose forward, so induced protein motion accumulates
`--no-memory-mode`	flag	`true`	transplant only the ligand coordinates each step
`--dock-zero`	flag		dock onto the path's first point before starting
`--param`	`KEY=VALUE`		override any parameters. *key (repeatable), e.g. --param rmsd_constraint_sd=1.5* (repeatable)*
`--preset`	`auto` \	`benchmark`	`auto`
`--no-large-ligand-rule`	flag		do not apply the paper's >50-atom settings (conformer_rmsd_threshold and outlier_rmsd both 5.0)

stage 2 · trajectory

option	value	default	what it does
`--traj-step`	`A`		window spacing (default: 0.5 ligand, 1.0 peptide)
`--traj-smooth`	`S`		spline smoothing (default: 0.5 ligand, 1.0 peptide)
`--traj-nstruct`	`N`	`10`	decoys per window (default: 10)
`--traj-path`	`FILE`		guide path to use (default: the coarse medoid)
`--run-stats`	flag		second scoring pass per window, i.e. error bars
`--nstruct-stats`	`N`	`10`	
`--noise`	flag		jitter the projected previous point, to escape local traps
`--outlier-rmsd`	`A`		
`--end-padding`	`A`		relax the in-plane restraint over the last windows
`--checkpoint-interval`	`N`	`10`	0 disables; the run resumes from checkpoint.pkl (default: 10)
`--use-tmp`	flag		run each window on a RAM disc and copy results back
`--tmp-path`	`DIR`		
`--fresh`	flag		ignore an existing checkpoint and start over
`--clean-on-fresh`	flag		with --fresh, also delete the old run directory
`--run-name`	`NAME`	`trajectory`	trajectory run subdirectory (default: trajectory)
`--path-per-replicate`	flag		give each replicate its own coarse path (from the coarse_NN runs) instead of all sharing the medoid. Samples different exit ROUTES rather than putting error bars on one route
`--traj-replicates`, `-R`	`N`	`1`	run N independent trajectories serially, into repl-00.. repl-NN, then pool them with `cluster`. On a cluster use the sbatch array instead, which runs them in parallel (default: 1)

`lpf_run.py analyze`

project

option	value	default	what it does
`--project`, `-o`	`DIR`		project directory: holds the prepared inputs and every run
`--n-proc`	`N`	`1`	MPI ranks per Rosetta call; >1 needs an .mpi. build and N <= nstruct (default: 1)
`--no-archive`	flag		do not zip each finished node/window directory
`--dry-run`	flag		prepare and preflight, but never call Rosetta
`--sec-per-decoy`	`S`	`16.0`	only for the runtime estimate. The default is implied by the paper's benchmark (2.7 min/frame at nstruct 10, one core, 136 systems); its spread is 0.9-8.2 min/frame, so recalibrate from your own first window (default: 16)
`--set`	`KEY=VALUE`		passed straight to LPF as --set (repeatable), e.g. --set output.use_tmp=true *(repeatable)*
`--verbose`, `-v`	flag		more detail from the preparation steps

analysis

option	value	default	what it does
`--run-dir`	`DIR`		trajectory run to analyse (default: the one just run)
`--plot`	`PNG`		save the energy profile (default name inside the run dir)
`--smooth-window`	`N`	`0`	also draw a running mean over N windows
`--csv`	`CSV`		write window, distance, interface and total score as CSV
`--zip`	flag		archive the analysis files
`--zip-windows`	flag		include every window directory in the archive (large)

`lpf_run.py run`

The whole pipeline in one go.

other

option	value	default	what it does
`--skip-coarse`	flag		go straight to stage 2 (needs --guide-path with 4+ points)

project

option	value	default	what it does
`--project`, `-o`	`DIR`		project directory: holds the prepared inputs and every run
`--n-proc`	`N`	`1`	MPI ranks per Rosetta call; >1 needs an .mpi. build and N <= nstruct (default: 1)
`--no-archive`	flag		do not zip each finished node/window directory
`--dry-run`	flag		prepare and preflight, but never call Rosetta
`--sec-per-decoy`	`S`	`16.0`	only for the runtime estimate. The default is implied by the paper's benchmark (2.7 min/frame at nstruct 10, one core, 136 systems); its spread is 0.9-8.2 min/frame, so recalibrate from your own first window (default: 16)
`--set`	`KEY=VALUE`		passed straight to LPF as --set (repeatable), e.g. --set output.use_tmp=true *(repeatable)*
`--verbose`, `-v`	flag		more detail from the preparation steps

environment

option	value	default	what it does
`--lpf-repo`	`DIR`		LigandPathFinder checkout (searched for if omitted)
`--clone`	flag		clone LigandPathFinder if it is not found

Rosetta

option	value	default	what it does
`--rosetta`	`PATH`		rosetta_scripts binary (searched for if omitted)
`--rosetta-database`	`DIR`		Rosetta database; found next to the binary if omitted. Written into the options file as -in:path:database
`--rosetta-backend`	`auto` \	`binary` \	`pyrosetta`
`--pyrosetta-python`	`PATH`		interpreter that has PyRosetta (default: this one)
`--install-pyrosetta`	flag		pip-install PyRosetta if absent (~1.5 GB download)
`--shim-jobs`	`N`	`1`	PyRosetta backend: worker processes per Rosetta call; each needs 1-2 GB RAM (default: 1)

target protein

option	value	default	what it does
`--target`, `-t`	`PDB	ID`		
`--target-chains`	`A,B`		protein chains to keep (default: all)
`--keep-hetero`	`HEM,ZN`		hetero residues to keep; each needs its own .params via --extra-params
`--keep-waters`	flag		keep water molecules
`--strip-hydrogens`	flag		drop hydrogens and let Rosetta rebuild them
`--extra-params`	`F.params`		cofactor params files, added to extra_files and to -extra_res_fa
`--auto-setup-metals`	flag		add -in:auto_setup_metals to the options file
`--header`	`FILE`		lines prepended to every task.pdb (metal constraints, membrane spans, symmetry)

ligand / peptide

option	value	default	what it does
`--mode`	`ligand` \	`peptide`	`ligand`
`--ligand`, `-l`	`FILE.sdf`		SD/MOL file whose FIRST entry is the bound pose
`--ligand-smiles`	`SMILES`		build from SMILES — no bound pose, so for testing only
`--ligand-dir`	`DIR`		directory that already holds lig.params + lig_conformers.pdb
`--ligand-pose`	`PDB`		with --ligand-dir: the ligand in its bound pose, atom names matching lig.params
`--ligand-pose-chain`	`CH`		if --ligand-pose is a whole complex, which chain in it is the ligand
`--peptide-chain`	`CH`		--mode peptide: chain of the bound peptide in --target
`--peptide-pdb`	`PDB`		--mode peptide: peptide-only PDB, in the receptor's frame
`--peptide-sequence`	`SEQ`		override the one-letter sequence read from the PDB
`--no-repair-terminus`	flag		do not build a missing terminal O/OXT
`--no-protonation`	flag		keep the neutral template instead of pH-7 charge states
`--ligand-chain`	`CH`	`X`	chain the ligand gets in start.pdb; must be free (default: X)
`--ligand-resnum`	`N`	`1`	residue number of the ligand in its chain (default: 1)
`--resname`	`ABC`		3-character PDB residue name (default: lig, or pep for --mode peptide)

conformer library

option	value	default	what it does
`--n-conformers`	`N`	`200`	ligand mode: ETKDG conformers to embed (default: 200)
`--n-local`	`N`	`120`	peptide mode: small perturbations of a few torsions
`--n-medium`	`N`	`120`	peptide mode: larger perturbations of more torsions
`--n-global`	`N`	`60`	peptide mode: free ETKDG embeddings
`--sigma-local`	`DEG`	`12.0`	
`--sigma-medium`	`DEG`	`45.0`	
`--prune-rmsd`	`A`		minimum RMSD between kept conformers (default: 0.5 ligand, 0.6 peptide)
`--no-minimize`	flag		skip the short MMFF relaxation of each conformer
`--kekulize`	flag		write a Kekule SD file; molfile_to_params expects explicit aromatic bonds, so leave this off
`--recharge`	`CHG`		force the net charge instead of taking it from the SD file's formal charges
`--conformer-report`	flag		report the library's RMSD ladder and whether it is connected
`--molfile-to-params`	`PATH`		Rosetta's molfile_to_params.py (searched for, then downloaded, if omitted)
`--no-download`	flag		never download molfile_to_params.py
`--seed`	`N`	`61453`	

tracked atom (NBR_ATOM)

option	value	default	what it does
`--nbr-atom`	`NAME`		params atom name to track, e.g. C17
`--nbr-atom-source`	`"CA 5"`		name it in the original peptide instead, e.g. "CA 5"
`--nbr-central`	flag		ligand mode: pick the most central atom rather than keeping molfile_to_params' choice (always done for peptides)
`--nbr-radius-margin`	`A`	`0.5`	

guide points

option	value	default	what it does
`--guide-path`	`FILE`		your own path (xyz text or PDB), e.g. from CAVER or AQUA-DUCT. With 4+ points you can skip stage 1. It must run from the binding site outwards
`--reverse-path`	flag		flip --guide-path, for a tunnel exported outside-in
`--exit-point`	`X,Y,Z`		exit point, instead of scanning for one
`--ray-index`	`N`	`0`	which ranked exit ray to take (default: 0, the widest)
`--list-rays`	`N`	`0`	print the top N exit rays and stop
`--hint-vector`	`X,Y,Z`		restrict the scan to a cone about this direction
`--hint-cone`	`DEG`	`60.0`	
`--scan-directions`	`N`	`512`	
`--scan-max-dist`	`A`	`50.0`	
`--exit-clearance`	`A`	`8.0`	how far from the protein the endpoint must sit (default: 8)
`--min-exit-distance`	`A`		path length floor (default: from the molecule's span)

start-pose minimisation (tutorial 1.3)

option	value	default	what it does
`--minimize-start`	flag		relax the assembled complex into the Rosetta force field before running LPF. Do this unless your start pose is already minimised — LPF assumes it is
`--minimize-nstruct`	`N`	`5`	independent minimisation attempts; the best by constraint-free total_score is kept (default: 5)
`--minimize-ca-sd`	`A`	`0.5`	C-alpha coordinate restraint sd; 0 reproduces the tutorial's unrestrained `minimize` run (default: 0.5)
`--no-minimize-backbone`	flag		side chains and rigid-body only

protocols

option	value	default	what it does
`--grid-width`	`A`		SCORINGGRIDS width (default: sized from the molecule)
`--transform-box`	`A`		Transform translation box (default: 5 ligand, 8 peptide)
`--no-tuning`	flag		copy the upstream protocols verbatim, only retargeting the chain

shared stage parameters

option	value	default	what it does
`--conformer-threshold`	`A`		conformer_rmsd_threshold; also switches subsampling on
`--memory-mode`	flag		carry the whole relaxed pose forward, so induced protein motion accumulates
`--no-memory-mode`	flag	`true`	transplant only the ligand coordinates each step
`--dock-zero`	flag		dock onto the path's first point before starting
`--param`	`KEY=VALUE`		override any parameters. *key (repeatable), e.g. --param rmsd_constraint_sd=1.5* (repeatable)*
`--preset`	`auto` \	`benchmark`	`auto`
`--no-large-ligand-rule`	flag		do not apply the paper's >50-atom settings (conformer_rmsd_threshold and outlier_rmsd both 5.0)

stage 1 · coarse

option	value	default	what it does
`--coarse-step`	`A`		node spacing (default: 2.0 ligand, 3.0 peptide)
`--coarse-nstruct`	`N`	`10`	decoys per node (default: 10)
`--replicates`, `-r`	`N`	`1`	independent coarse runs; the medoid path goes forward (default: 1)
`--jitter-radius`	`A`		endpoint jitter radius (default: 5)
`--keep-previous`	flag		do not wipe an existing coarse directory (the coarse stage cannot resume, so re-runs normally need a clean one)

stage 2 · trajectory

option	value	default	what it does
`--traj-step`	`A`		window spacing (default: 0.5 ligand, 1.0 peptide)
`--traj-smooth`	`S`		spline smoothing (default: 0.5 ligand, 1.0 peptide)
`--traj-nstruct`	`N`	`10`	decoys per window (default: 10)
`--traj-path`	`FILE`		guide path to use (default: the coarse medoid)
`--run-stats`	flag		second scoring pass per window, i.e. error bars
`--nstruct-stats`	`N`	`10`	
`--noise`	flag		jitter the projected previous point, to escape local traps
`--outlier-rmsd`	`A`		
`--end-padding`	`A`		relax the in-plane restraint over the last windows
`--checkpoint-interval`	`N`	`10`	0 disables; the run resumes from checkpoint.pkl (default: 10)
`--use-tmp`	flag		run each window on a RAM disc and copy results back
`--tmp-path`	`DIR`		
`--fresh`	flag		ignore an existing checkpoint and start over
`--clean-on-fresh`	flag		with --fresh, also delete the old run directory
`--run-name`	`NAME`	`trajectory`	trajectory run subdirectory (default: trajectory)
`--path-per-replicate`	flag		give each replicate its own coarse path (from the coarse_NN runs) instead of all sharing the medoid. Samples different exit ROUTES rather than putting error bars on one route
`--traj-replicates`, `-R`	`N`	`1`	run N independent trajectories serially, into repl-00.. repl-NN, then pool them with `cluster`. On a cluster use the sbatch array instead, which runs them in parallel (default: 1)

analysis

option	value	default	what it does
`--run-dir`	`DIR`		trajectory run to analyse (default: the one just run)
`--plot`	`PNG`		save the energy profile (default name inside the run dir)
`--smooth-window`	`N`	`0`	also draw a running mean over N windows
`--csv`	`CSV`		write window, distance, interface and total score as CSV
`--zip`	flag		archive the analysis files
`--zip-windows`	flag		include every window directory in the archive (large)

`lpf_run.py cluster`

Cluster finished trajectory runs by path similarity and average their energy profiles per cluster, as the paper does for an ensemble of replicates.

other

option	value	default	what it does
`--project`, `-o`	`DIR`		take repl-/ *or trajectory/* from this project instead
`--ligand-chain`	`CH`		default: from the project manifest, else X
`--min-cluster-size`	`N`	`10`	discard clusters with this many members or fewer (default: 10, the paper's value)
`--missing-penalty`	`A`	`0.5`	distance charged per missing frame (default: 0.5, the paper's value)
`--frame-step`	`A`		per-frame spacing for the x-axis (default: from run.json)
`--plot`	`PNG`	`true`	
`--csv`	`CSV`		
`--out`	`DIR`		where to write outputs
`--convergence`	flag		also measure how much the pooled mean still moves as replicates are added, to judge whether you have enough
`--verbose`, `-v`	flag		

`lpf_run.py rescore`

Rescore each window's selected pose with a scoring-only protocol. Use this when interface_scores.txt came out all zeros: it costs one Rosetta call per window instead of nstruct docking runs.

other

option	value	default	what it does
`--project`, `-o`	`DIR`		
`--run-dir`	`DIR`		default: the trajectory run in the manifest
`--protocol`	`XML`		default: zero_score.xml from the project's files/
`--rosetta`	`PATH`		default: the binary recorded in the manifest
`--out-name`	`NAME`	`interface_scores_rescored.txt`	written next to the original, which is left untouched
`--install`	flag		replace interface_scores.txt with the rescored values (the original is backed up first), so analyze and cluster use them
`--keep-work`	flag		keep the per-window working directories

`lpf_run.py doctor`

environment

option	value	default	what it does
`--lpf-repo`	`DIR`		LigandPathFinder checkout (searched for if omitted)
`--clone`	flag		clone LigandPathFinder if it is not found

Rosetta

option	value	default	what it does
`--rosetta`	`PATH`		rosetta_scripts binary (searched for if omitted)
`--rosetta-database`	`DIR`		Rosetta database; found next to the binary if omitted. Written into the options file as -in:path:database
`--rosetta-backend`	`auto` \	`binary` \	`pyrosetta`
`--pyrosetta-python`	`PATH`		interpreter that has PyRosetta (default: this one)
`--install-pyrosetta`	flag		pip-install PyRosetta if absent (~1.5 GB download)
`--shim-jobs`	`N`	`1`	PyRosetta backend: worker processes per Rosetta call; each needs 1-2 GB RAM (default: 1)

`lpf_run.py demo`

other

option	value	default	what it does
`--demo-points`	`N`	`6`	how many of the demo's guide points to use (default: 6, a short run)

project

option	value	default	what it does
`--project`, `-o`	`DIR`		project directory: holds the prepared inputs and every run
`--n-proc`	`N`	`1`	MPI ranks per Rosetta call; >1 needs an .mpi. build and N <= nstruct (default: 1)
`--no-archive`	flag		do not zip each finished node/window directory
`--dry-run`	flag		prepare and preflight, but never call Rosetta
`--sec-per-decoy`	`S`	`16.0`	only for the runtime estimate. The default is implied by the paper's benchmark (2.7 min/frame at nstruct 10, one core, 136 systems); its spread is 0.9-8.2 min/frame, so recalibrate from your own first window (default: 16)
`--set`	`KEY=VALUE`		passed straight to LPF as --set (repeatable), e.g. --set output.use_tmp=true *(repeatable)*
`--verbose`, `-v`	flag		more detail from the preparation steps

environment

option	value	default	what it does
`--lpf-repo`	`DIR`		LigandPathFinder checkout (searched for if omitted)
`--clone`	flag		clone LigandPathFinder if it is not found

Rosetta

option	value	default	what it does
`--rosetta`	`PATH`		rosetta_scripts binary (searched for if omitted)
`--rosetta-database`	`DIR`		Rosetta database; found next to the binary if omitted. Written into the options file as -in:path:database
`--rosetta-backend`	`auto` \	`binary` \	`pyrosetta`
`--pyrosetta-python`	`PATH`		interpreter that has PyRosetta (default: this one)
`--install-pyrosetta`	flag		pip-install PyRosetta if absent (~1.5 GB download)
`--shim-jobs`	`N`	`1`	PyRosetta backend: worker processes per Rosetta call; each needs 1-2 GB RAM (default: 1)

shared stage parameters

option	value	default	what it does
`--conformer-threshold`	`A`		conformer_rmsd_threshold; also switches subsampling on
`--memory-mode`	flag		carry the whole relaxed pose forward, so induced protein motion accumulates
`--no-memory-mode`	flag	`true`	transplant only the ligand coordinates each step
`--dock-zero`	flag		dock onto the path's first point before starting
`--param`	`KEY=VALUE`		override any parameters. *key (repeatable), e.g. --param rmsd_constraint_sd=1.5* (repeatable)*
`--preset`	`auto \`	`benchmark`	`auto`
`--no-large-ligand-rule`	flag		do not apply the paper's >50-atom settings (conformer_rmsd_threshold and outlier_rmsd both 5.0)

stage 2 · trajectory

option	value	default	what it does
`--traj-step`	`A`		window spacing (default: 0.5 ligand, 1.0 peptide)
`--traj-smooth`	`S`		spline smoothing (default: 0.5 ligand, 1.0 peptide)
`--traj-nstruct`	`N`	`10`	decoys per window (default: 10)
`--traj-path`	`FILE`		guide path to use (default: the coarse medoid)
`--run-stats`	flag		second scoring pass per window, i.e. error bars
`--nstruct-stats`	`N`	`10`	
`--noise`	flag		jitter the projected previous point, to escape local traps
`--outlier-rmsd`	`A`		
`--end-padding`	`A`		relax the in-plane restraint over the last windows
`--checkpoint-interval`	`N`	`10`	0 disables; the run resumes from checkpoint.pkl (default: 10)
`--use-tmp`	flag		run each window on a RAM disc and copy results back
`--tmp-path`	`DIR`		
`--fresh`	flag		ignore an existing checkpoint and start over
`--clean-on-fresh`	flag		with --fresh, also delete the old run directory
`--run-name`	`NAME`	`trajectory`	trajectory run subdirectory (default: trajectory)
`--path-per-replicate`	flag		give each replicate its own coarse path (from the coarse_NN runs) instead of all sharing the medoid. Samples different exit ROUTES rather than putting error bars on one route
`--traj-replicates`, `-R`	`N`	`1`	run N independent trajectories serially, into repl-00.. repl-NN, then pool them with `cluster`. On a cluster use the sbatch array instead, which runs them in parallel (default: 1)

analysis

option	value	default	what it does
`--run-dir`	`DIR`		trajectory run to analyse (default: the one just run)
`--plot`	`PNG`		save the energy profile (default name inside the run dir)
`--smooth-window`	`N`	`0`	also draw a running mean over N windows
`--csv`	`CSV`		write window, distance, interface and total score as CSV
`--zip`	flag		archive the analysis files
`--zip-windows`	flag		include every window directory in the archive (large)

`lpf_run.py relocate`

Every path this tooling writes is absolute, so moving a project leaves stale paths in project.json, each run.json, the checkpoints and the PyRosetta launcher. This rewrites them to the directory's new location.

other

option	value	default	what it does
`--project`, `-o`	`DIR`		the project in its NEW location
`--old-root`	`PATH`		the old project path; read from project.json otherwise
`--remap`	`OLD=NEW`		extra path mapping for files that lived outside the project, e.g. the --target structure (repeatable) *(repeatable)*
`--dry-run`	flag		report what would change and write nothing

---

12. Further reading

- `README.md` next to this file — the notebooks, the PyRosetta backend, the SI cross-check, and what has and has not been verified.
- `docs/config_reference.md` in the LigandPathFinder checkout — every `run.json` key, and `docs/unused_config_settings.md` for the keys each stage ignores.
- `demo/tutorial.md` upstream — the original worked walkthrough, including the retuning checklist in its §5.
- The paper's SI — the coarse-path algorithm, the benchmark settings and timings, and the clustering method.
- Reference runs and the paper's dataset: 10.5281/zenodo.23183178.

Licences. This script and its helpers: use freely. LPF is MIT (its own code only). **Rosetta is licensed separately** — that covers PyRosetta, `molfile_to_params.py`, and the `rosetta_py` modules the script can download.